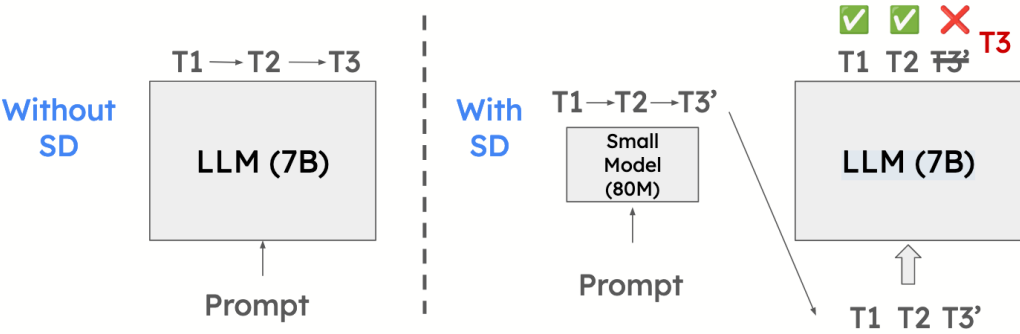


Speculative Decoding 推测解码方案详解

当前，大型语言模型（LLM）在推理阶段普遍采用自回归解码策略，其核心特性是**逐步串行生成 token，每一步都依赖前一步的输出**。这一计算模式导致推理过程在系统层面面临严重的**内存带宽瓶颈**：每一步前向计算都需要将完整的模型参数从**高带宽内存（HBM）加载到加速器缓存**，但仅生成一个 token。由于每次只生成一个 token，导致大量的计算资源被闲置，无法充分发挥加速器的算力潜力，最终造成整体推理效率低下。

为解决这一问题，一种加速大型语言模型推理的思路是**提高解码过程的算术强度**（即总浮点运算次数 FLOPs 与数据传输量之间的比值），同时**减少解码步骤**。基于这一理念，研究者们提出了**推测解码/投机解码（Speculative Decoding）**技术。Speculative Decoding 的核心思路如下图所示，首先以低成本的方式（一般来说是用小模型）快速生成多个候选 token，然后通过一次并行验证阶段快速验证多个 token，进而减少大模型的 decode 次数，从而达到加速的目的。



这种方法在实际中之所以有效，是因为大多数时候草稿模型生成的 token 都会被接受。这些 token 本身就容易预测，即使是一个小得多的草稿模型也能准确生成。当这些容易的 token 被接受时，模型可以快速跳过这些部分。对于那些较难预测的 token，如果目标大模型不同意草稿模型的输出，就会退回到原始的解码速度，甚至略微变慢，因为还需要进行额外的验证计算。

这里举两个例子：

比如 prompt 是 `The capital of South Korea is ?`，那么输出大概率是 `The capital of` 开头，这些 token 都很常见，草稿模型很容易预测正确，因此可以被目标模型接受，快速跳过。

在代码提示的场景下，也很好预测，比如 prompt 是：

```
循环以下数组：
nums = [1, 2, 3]
```

输出大概率是：

```
for num in nums:
    xxx
```

1 在 vLLM 中使用 Speculative Decoding

在 vLLM 中可以通过 `speculative_config` 参数来使用 Speculative Decoding。以下代码通过离线模式使用草稿模型进行 Speculative Decoding，每次推测 5 个 token。

```
from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="facebook/opt-6.7b",
    tensor_parallel_size=1,
    speculative_config={
        "model": "facebook/opt-125m",
        "num_speculative_tokens": 5,
    },
)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

使用以下命令可以以在线模式运行 Speculative Decoding。

```
vllm serve \
facebook/opt-6.7b \
--speculative-config '{"model": "facebook/opt-125m", "num_speculative_tokens": 5}'
```

2 早期的 Speculative Decoding

论文：

Fast Inference from Transformers via Speculative Decoding: <https://arxiv.org/abs/2211.17192>

Accelerating Large Language Model Decoding with Speculative Sampling: <https://arxiv.org/abs/2302.01318>

Speculative Decoding 最早源自 Google 在 2023 年发表的一篇论文，标题为 "Fast Inference from Transformers via Speculative Decoding"，该论文首次提出了通过“草稿模型 + 验证模型”的方式来加速 Transformer 模型的推理。

同一时期，DeepMind 也独立发布了另一篇相关论文 "Accelerating Large Language Model Decoding with Speculative Sampling"，其背后的核心思想与 Google 的这篇类似。

下图展示了 Speculative Decoding 算法的执行流程。

Algorithm 2 Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead K and minimum target sequence length T . 每轮草稿生成 K 个 token，目标总生成长度 T

Given auto-regressive target model $q(\cdot|\cdot)$, and auto-regressive draft model $p(\cdot|\cdot)$, initial prompt sequence $x_0, \dots, x_t$. 给定两个模型：
- 目标模型 q (大模型，准确) 初始 prompt 序列 x_0 到 x_t
- 草稿模型 p (小模型，快)

Initialise $n \leftarrow t$.

while $n < T$ **do** 当生成的 token 数量还没达到目标 T ，就持续生成

for $t = 1 : K$ **do** 用草稿模型 p 自回归地生成 K 个 token，每个 $\tilde{x}_t$ 都依赖已有上下文和前面生成的草稿 token。

 Sample draft auto-regressively $\tilde{x}_t \sim p(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_{t-1})$

end for

In parallel, compute $K + 1$ sets of logits from drafts $\tilde{x}_1, \dots, \tilde{x}_K$: 并行地用目标模型 q 计算 $K+1$ 个位置上的 logits，分别用于验证草稿中的每个 token，再额外预测 1 个 bonus token

$q(x|x_1, \dots, x_n)$ 是一个向量，表示目标模型在给定上下文下，对词表中所有可能下一个 token 的打分 (logits)，维度等于词表大小。
 $q(x|x_1, \dots, x_n), q(x|x_1, \dots, x_n, \tilde{x}_1), \dots, q(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_K)$

for $t = 1 : K$ **do**

 Sample $r \sim U[0, 1]$ from a uniform distribution. 从 $[0, 1]$ 的均匀分布中随机采一个数 r

if $r < \min\left(1, \frac{q(x|x_1, \dots, x_{n+t-1})}{p(x|x_1, \dots, x_{n+t-1})}\right)$ **then** 如果 r 小于 q 和 p 的比值 (跟草稿模型的预测越接近，越容易被接受)，接受该草稿 token

 Set $x_{n+t} \leftarrow \tilde{x}_t$ and $n \leftarrow n + 1$. 将草稿 token 加入最终生成的序列，并更新序列的长度

else 否则拒绝草稿 token，从目标模型中重新采样一个 token。
 使用 $q-p$ 中的正值部分 (表示草稿没覆盖的部分)，然后中断整个草稿 token 验证过程。
 sample $x_{n+t} \sim (q(x|x_1, \dots, x_{n+t-1}) - p(x|x_1, \dots, x_{n+t-1}))_+$ and exit for loop.

end if

end for 如果草稿模型预测的所有 token 都接受了，额外生成一个 bonus token

 If all tokens $x_{n+1}, \dots, x_{n+K}$ are accepted, sample extra token $x_{n+K+1} \sim q(x|x_1, \dots, x_n, x_{n+K})$ and set $n \leftarrow n + 1$.

end while

为了更直观地理解 Speculative Decoding 算法的执行流程，我们以下面的场景为例，走一遍整个生成过程。

假设当前的上下文为: "This apple"，词表如下：

```
vocab = ["This", "apple", "is", "very", "delicious", "bad", "today"]
```

本轮草稿长度 $K=2$ ，也就是说草稿模型 p 将尝试生成两个 token。

第一步：草稿模型生成 token

草稿模型 p 是一个轻量的自回归模型。每一步会计算 logits，并通过 softmax 得到概率分布，从中采样出下一个 token。草稿模型对每个生成 token 的概率 (即 softmax 后的值) 会被保留，用于之后目标模型的验证过程。

```
# 基于上下文 "This apple"，预测出下一个 token:
p_logits_1 = [1.5, 1.8, 2.5, 1.1, 0.3, 0.05, -1.0] # 草稿模型 logits for "This apple"
p_probs_1 = softmax(p_logits_1) # 草稿模型计算 softmax 概率
# → 草稿模型生成:  $\tilde{x}_1 = "is"$ 

# 接着以 "This apple is" 为上下文继续预测:
p_logits_2 = [0.7, 0.9, 1.0, 1.3, 0.4, 0.1, -0.8] # 草稿模型 logits for "This apple is"
p_probs_2 = softmax(p_logits_2) # 草稿模型计算 softmax 概率
# → 草稿模型生成:  $\tilde{x}_2 = "very"$ 
```

第二步：目标模型并行计算 logits

目标模型 q 是较大但更精确的模型，得益于草稿 token 已经生成完毕，可以**并行地**对草稿序列的每个位置进行打分（即计算 logits），从而加快验分过程。

```
q_logits_1 = [1.8, 2.0, 2.2, 1.2, 0.5, 0.1, -0.7] # 目标模型 q(x | "This apple")
q_logits_2 = [0.9, 1.1, 1.0, 1.1, 0.7, 0.2, -0.5] # q(x | "This apple is")
q_logits_3 = [0.5, 0.4, 1.2, 0.7, 2.5, -0.2, 0.0] # q(x | "This apple is very")

q_probs_1 = softmax(q_logits_1)
q_probs_2 = softmax(q_logits_2)
```

第三步：目标模型对草稿 token 验分

对第一个草稿 token： $\vec{x}_1 = \text{"is"}$ ，分别查找两个模型计算出的 softmax 概率：

```
# 提取 token "is" 对应的概率：
p_prob = p_probs_1["is"] # 草稿模型计算的概率
q_prob = q_probs_1["is"] # 目标模型计算的概率

# 假设：
p_prob = 0.30
q_prob = 0.22
```

计算接受概率：

$$\text{acceptprob}_1 = \min\left(1, \frac{qprob}{pprob}\right) = \min\left(1, \frac{0.22}{0.30}\right) = 0.733$$

假设采样出的随机数 $r_1 = 0.42$ ，因为：

$$0.42 < 0.733 \Rightarrow \text{接受!is!}$$

对第二个草稿 token： $\vec{x}_2 = \text{"very"}$ ：

```
p_prob = p_probs_2["very"] = 0.28 # 草稿模型计算的概率
q_prob = q_probs_2["very"] = 0.24 # 目标模型计算的概率
```

$$\text{acceptprob}_2 = \min\left(1, \frac{0.24}{0.28}\right) = 0.857$$

随机数 $r_2 = 0.62$ ，因为：

$$0.62 < 0.857 \Rightarrow \text{接受!very!}$$

第四步：生成奖励 token (bonus token)

因为草稿中的所有 token 都通过了目标模型的验分，我们使用事先计算好的：

```
q(x | "This apple is very") → q_logits_3
```

对其进行 softmax 操作，得到概率分布。假设采用贪心解码策略，则选择概率最大的 token。

```
q_logits_3 = [0.5, 0.4, 1.2, 0.7, 2.5, -0.2, 0.0]
# softmax 后 "delicious" 概率最大
```

因此生成奖励 token：

```
x_{n+k+1} = "delicious"
```

3 草稿模型的限制

虽然 Speculative Decoding 通过“猜测-验证”策略，在多个场景下显著提升了解码速度，但草稿模型仍然存在以下限制：

- **接受率受限，加速效果不稳定**：草稿模型的预测结果必须通过大模型的验证才能被接受。加速效果高度依赖 token 的接受率 (Acceptance Rate)。一旦接受率较低，不仅无法提升性能，反而会因为重复验证和回退而拖慢整体解码速度。
- **难以获得“又小又准”的模型**：要让草稿模型既轻量又能精准模拟大模型的行为非常困难。现实中经常出现 distribution shift（分布偏移），即草稿模型与大模型输出之间存在差异，导致预测失败率上升。
- **训练和泛化困难**：很多 LLM 并没有现成的草稿模型，因此需要重新训练一个额外模型，这带来了显著的成本。而且训练出的模型往往无法泛化到其他基础模型或数据集上，需要频繁地调优和维护。
- **系统架构复杂度上升**：同时维护两个模型（草稿模型 + 目标模型）会增加推理系统的复杂性，特别是在分布式部署中，需要解决模型调度、资源隔离和负载均衡等工程问题。

4 Prompt Lookup Decoding

在许多大语言模型（LLM）的应用场景中，例如摘要生成、文档问答、多轮对话和代码编辑，模型的输入（prompt）与输出之间往往存在大量的 **n-gram（连续 token 片段）重合**。这些重合内容通常是实体名称、常见短语或代码片段，模型在生成时常常直接从输入中复制它们。

Prompt Lookup Decoding 正是利用了这一规律，在生成过程中加速自回归解码。它通过高效的字符串匹配算法（如 KMP）快速定位匹配位置，直接利用 prompt 中已有的信息进行预测，而**无需依赖额外的草稿模型**生成候选 token。尤其在模型输出高度重复 prompt 内容的场景下，这种方法能显著提升推理效率。当大模型在答案中重复 prompt 的内容时，这种方法效果尤为显著。

vLLM 的 n-gram Speculative Decoding 不仅仅使用原始提示进行匹配，而是使用整个上下文序列（包括原始提示和已生成的标记）。每次生成新标记后，整个上下文序列都会更新，然后用于下一轮的 n-gram 匹配。

● Prompt: What is the capital of South Korea?

2-gram	3 speculate tokens	During generation: the capital of South Korea ✓ ✓ ✓
What is	the capital of	
is the	capital of South	
the capital	of South Korea	
capital of	South Korea ?	

上图展示了一个 Prompt Lookup Decoding 的示例：给定一个 prompt，我们会提取其中所有的 2-gram 作为查找键（key），并将它们后面紧跟的三个 token 作为查找值（value）。在生成过程中，我们会检查当前生成的 2-gram 是否与查找表中的某个 key 匹配。如果匹配成功，就使用对应的 value 作为后续的候选 token 进行生成。

在下面这段代码中，我们使用了 n-gram Speculative Decoding，每次最多推测 3 个 token，最多使用 2 个 n-gram 进行匹配。

```
from vllm import LLM, SamplingParams

prompts = [
    "what is the capital of South Korea?",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="facebook/opt-6.7b",
    tensor_parallel_size=1,
    speculative_config={
        "method": "ngram", # 使用 n-gram Speculative Decoding
        "num_speculative_tokens": 3, # 每次最多推测 3 个 token
        "prompt_lookup_max": 2, # 最多使用 2 个 n-gram 进行匹配
    },
)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

5 Jacobi Decoding

论文：Accelerating Transformer Inference for Translation via Parallel Decoding: <https://arxiv.org/pdf/2305.10427>

Jacobi 迭代法是一种经典的非线性方程组求解方法。在大语言模型（LLM）推理中，我们也可以将其应用于**并行生成 token**，且**不依赖草稿模型**。传统上，自回归解码被视为一个**逐步生成 token 的串行过程**，如下图左侧所示。而通过对一些公式进行简单重排，这一过程也可以被看作是在**求解一个非线性方程组**，如下图右侧所示。

$\mathbf{x}$: prompt, $\mathbf{y} = [y_1, y_2, \dots, y_m]$: m tokens to decode, $p(\mathbf{y}|\mathbf{x})$: LLM distribution

Define: $f(y_i, \mathbf{y}_{1:i-1}, \mathbf{x}) = y_i - \operatorname{argmax} p(y_i | \mathbf{y}_{1:i-1}, \mathbf{x})$

$$\begin{cases} y_1 = \operatorname{argmax} p(y_1 | \mathbf{x}) \\ y_2 = \operatorname{argmax} p(y_2 | y_1, \mathbf{x}) \\ \vdots \\ y_m = \operatorname{argmax} p(y_m | \mathbf{y}_{1:m-1}, \mathbf{x}) \end{cases} = \begin{cases} f(y_1, \mathbf{x}) = 0 \\ f(y_2, y_1, \mathbf{x}) = 0 \\ \vdots \\ f(y_m, \mathbf{y}_{1:m-1}, \mathbf{x}) = 0 \end{cases}$$

Autoregressive decoding

Non-linear system with
 m variables and m equations

通过 Jacobi 迭代法，可以并行地求解该非线性系统中的所有变量 $\mathbf{y} = [y_1, y_2, \dots, y_m]$ ，步骤如下：

- 为所有变量 $\mathbf{y} = [y_1, y_2, \dots, y_m]$ 设定一个初始猜测；
- 使用当前的 $\mathbf{y}$ 值，为每个方程计算新的 $\mathbf{y}$ ；
- 将 $\mathbf{y}$ 更新为新计算得到的 $\mathbf{y}$ ；
- 重复该过程，直到满足某个停止条件（例如 $\mathbf{y} = \mathbf{y}'$ ）为止。

Jacobi Decoding 之所以可以让大模型并行预测多个 token，是因为它将原本自回归的串行生成过程转化为了一个非线性系统的“并行迭代求解”问题。每一轮都用大模型，在不同位置上并行预测 token，并使用上一轮的结果作为输入，而不是依赖本轮刚生成的内容。



上图展示了这一并行解码过程（也称为 Jacobi Decoding）。Jacobi Decoding 能够在最多 m 步内求解所有 m 个变量（即，与自回归解码所需步数相同），因为每一步至少能够保证第一个 token 被正确解码。有时候，多个 token 可能会在一次迭代中同时收敛，从而减少整体解码步数。例如，Jacobi Decoding 在第 4 步中同时预测并接受了两个 token：“computer”和“scientist”。

与自回归解码相比，Jacobi Decoding 的每一步在计算上会稍微更昂贵一些，因为它需要在多个 token 上同时进行语言模型的前向计算。但幸运的是，由于 GPU 的并行处理特性，这种额外计算通常不会带来显著的延迟。

Jacobi Decoding 就像：你先把整篇文章草拟出来，然后让老师一段段检查、划掉不对的、保留正确的，然后继续在此基础上写后面的内容。

在实际应用中，Jacobi Decoding 在实现显著的实际加速方面面临不少挑战。尽管它在多轮迭代中确实能够一次生成多个 token，但这些 token 通常难以被准确地放置在正确的位置。即使部分 token 被正确预测，它们也常常在后续的迭代中被新的预测所替代。最终，真正能做到多个 token 同时生成且位置正确的情况非常有限，难以体现并行 Jacobi Decoding 原本追求的效率优势。

6 Lookahead Decoding

论文：Break the Sequential Dependency of LLM Inference Using Lookahead Decoding: <https://arxiv.org/abs/2402.02057>

Github: <https://github.com/hao-ai-lab/LookaheadDecoding>

Lookahead Decoding 的灵感来源于 Jacobi Decoding，它将自回归解码视为求解非线性系统的问题，并通过固定点迭代（fixed-point iteration）一次性并行生成多个未来 token。虽然 Jacobi Decoding 中初始化的 token 往往不准确，但其生成轨迹（Jacobi trajectory）中包含的 n -gram 片段可能在后续解码中成为有效的候选。

Lookahead Decoding 正是利用了这一特性：**收集 Jacobi 生成路径中的 n -gram，并将其缓存至 n -gram 池中**。随后系统在解码时从中挑选可能命中的 n -gram 并通过主模型验证是否可接受，从而跳跃式推进生成过程，显著降低推理延迟。

整个过程中，Lookahead Decoding 不依赖草稿模型，部署简洁，同时借助前瞻生成与并行验证两个分支，在单步内完成多个候选的生成与验证，最大化利用了原本未被自回归解码充分使用的计算资源。



7 Medusa

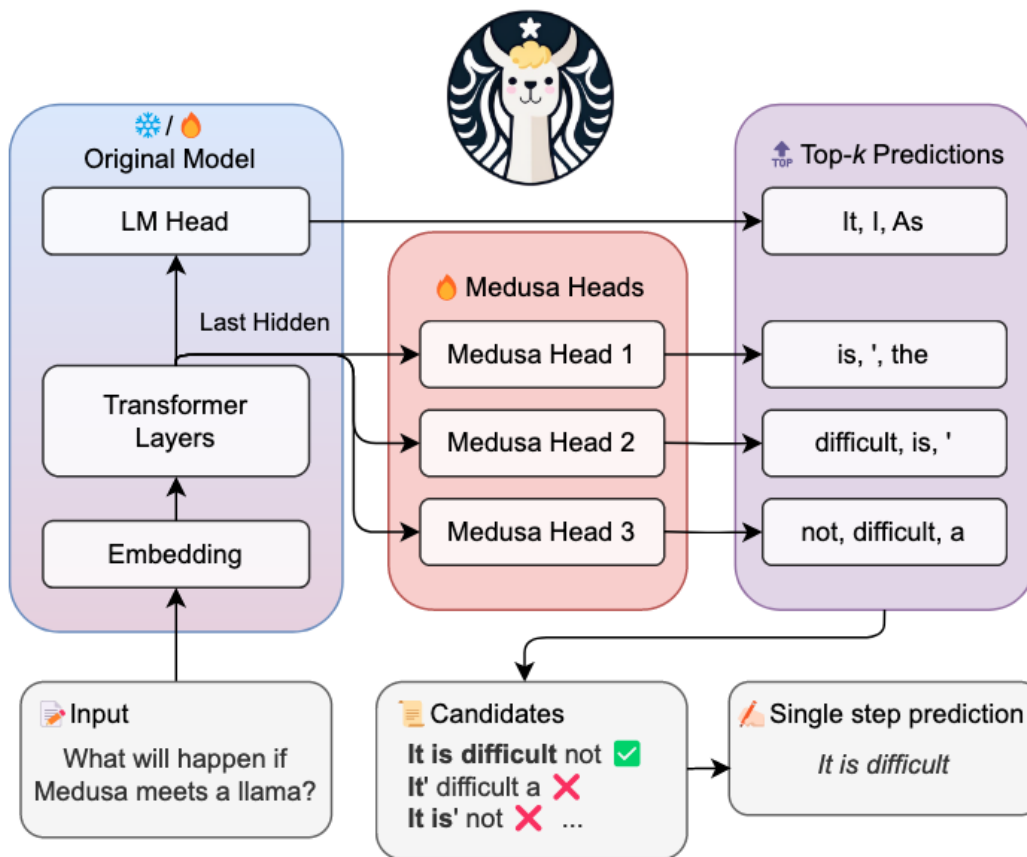
论文：Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads: <https://arxiv.org/abs/2401.10774>

Github: <https://github.com/FasterDecoding/Medusa>

相比于引入一个独立的草稿模型，**Medusa** 提出了一种更紧凑且高效的设计思路：直接在原始主干模型上进行扩展。其核心做法是在主模型的最后一个隐藏层上添加多个轻量级解码头（Medusa Heads），每个解码头并行预测未来不同位置的 token。Medusa Heads 是在原始主干模型（backbone model）基础上一同训练得到的。

每个预测头的结构极其简洁，仅由一层构成，与主干模型的输出层相似，因此不会增加推理服务系统的复杂度。同时，预测头的输出分布与主模型高度一致，有效缓解了草稿模型常见的分布偏移问题，显著提升了候选 token 的准确性与稳定性。

训练时，主干模型可以保持冻结状态（称为 MEDUSA-1），也可以与预测头一起联合训练（称为 MEDUSA-2）。这种设计使得即使只使用单个 GPU，也能对大型模型进行高效微调，充分利用主干模型已学到的强大特征表示。



在推理阶段，每个预测头会针对其对应的位置生成多个 top 候选 token。然后，这些候选会被组合成多个候选序列，并通过**树状注意力机制 (tree attention)** 并行处理。最后一步是采用典型接受策略 (typical acceptance) 来筛选出合理的生成路径，并将被接受的最长候选前缀作为下一轮解码的起点。通过同时接受更多的 token，可以减少所需的解码步数，从而提升整个解码过程的效率。

接下来，让我们深入了解 Medusa 的 3 个组成部分：Medusa heads、tree attention 和 typical acceptance。

7.1 Medusa heads

Medusa 之所以能够一次性解码多个 token，核心在于它在原始语言模型的**最后隐藏层输出之上，附加了多个解码头 (Medusa heads)**。每个解码头负责预测序列中不同偏移位置的 future token。这些解码头结构类似于原始模型的语言模型头，通常是单层前馈神经网络，可以独立地为其目标位置生成 token 预测。这种设计使得模型在一次前向传播中即可**并行预测多个后续 token**，而不再像传统自回归生成方式那样必须逐步生成单个 token，从而显著提升解码效率。

具体来说：

- 原始 LM head：预测当前位置 t 的下一个 token，即位置 $t+1$ ；
- 第 1 个 Medusa head：预测位置 $t+2$ ；
- 第 2 个 Medusa head：预测位置 $t+3$ ；
- ...
- 第 k 个 Medusa head：预测位置 $t+k+1$ 。

为了使 MEDUSA 的多个解码头具备良好的预测能力，需要对其进行训练。根据不同的应用场景和资源条件，可以选择不同的训练方式：

MEDUSA-1: 冻结主干，仅训练解码头

在该方案中，原始模型的主干（包括原有的解码头）保持冻结，仅训练新增的 MEDUSA 解码头。适用于计算资源有限，或希望完全保留原模型性能的场景。

该方式还可以结合 QLoRA 对解码头进行参数高效微调，从而进一步降低内存和计算资源的消耗。

实验表明，MEDUSA-1 在 Vicuna-7B 上可实现 **2.18× 的推理速度提升**。

MEDUSA-2: 联合训练主干与解码头

该方案对原模型主干和 MEDUSA 解码头进行联合训练。相比于 MEDUSA-1，虽然资源开销更大，但也能更充分地发挥多个解码头在推理加速方面的潜力。

MEDUSA-2 通过联合训练，使 MEDUSA 解码头的分布与主干模型保持一致，**有效缓解分布漂移问题，显著提升预测准确性**。

适用于计算资源充足，或从 base 模型进行 SFT（监督微调）的场景。实验证明，MEDUSA-2 在 Vicuna-7B 上可实现 **2.83× 的速度提升**，优于 MEDUSA-1。

为保证训练效果和稳定性，MEDUSA-2 引入了以下关键训练策略：

- 1. 差异化学习率 (Differential Learning Rates)

由于主干模型已经预训练充分、较为稳定，而 MEDUSA 解码头是新增模块、仍处于早期学习阶段，因此为两者设置不同的学习率是合理的选择：

- 主干模型使用较小的学习率，以避免扰动已有能力；
- MEDUSA 解码头使用较大的学习率，以加快其收敛速度。

这样既能提升训练效率，又能保护主干模型不被破坏。

- 2. 两阶段训练流程 (Two-Stage Training)

在训练初期，MEDUSA heads 的损失较大，可能导致梯度过大，从而干扰主干模型的参数更新。为此，采用两阶段训练流程：

- **第一阶段：**仅训练 MEDUSA 解码头，相当于执行 MEDUSA-1；
- **第二阶段：**再与主干模型一起联合训练。

具体可采用两种方式：

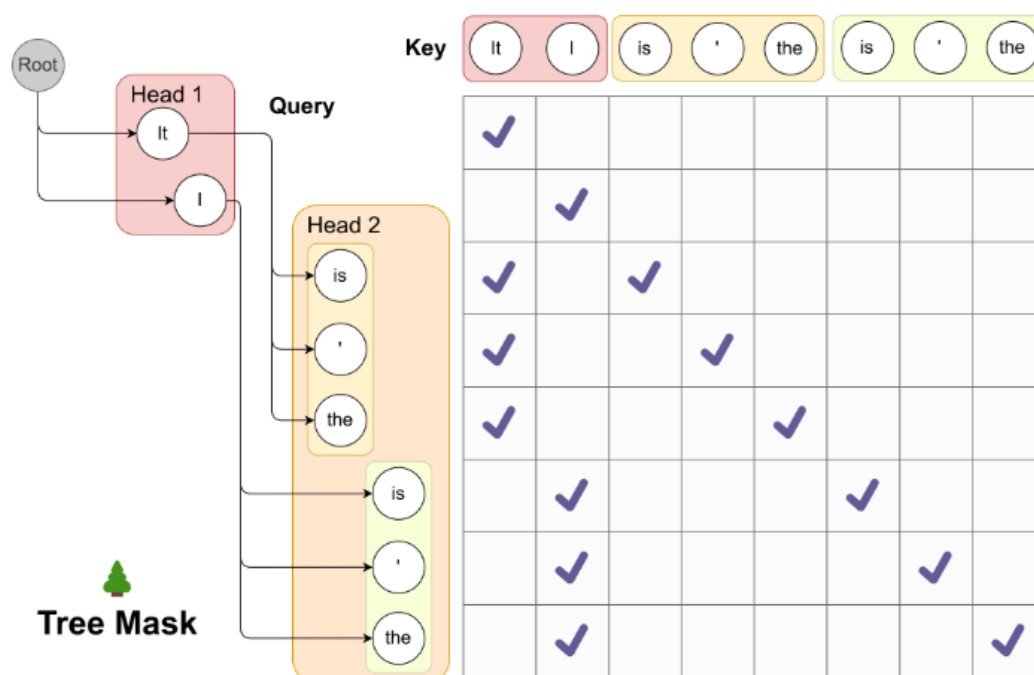
- 先单独训练主干模型若干 epoch，然后再与解码头联合训练；
- 或者使用更精细的 warmup 策略：**逐步增加 λ_0 (主干模型损失在总损失中的权重)**，在训练初期让主干模型“参与较少”，后期逐步“参与更多”。

训练数据的获取

- 如果原模型使用的 SFT 数据集可用，则可以直接用于训练 MEDUSA 解码头；
- 如果该数据集不可获得，或者原模型经过了 RLHF (人类反馈强化学习) 阶段，**则可以通过自蒸馏 (self-distillation) 方式生成训练数据**。自蒸馏的做法是利用模型自身生成符合其输出分布的样本，构建与原模型一致的数据分布，从而训练出与主干模型行为一致的 MEDUSA heads。

7.2 Tree Attention

Medusa 在推理时，每个解码头不仅输出一个最可能的 token，而是采样多个 top-k 候选 token，并将它们组合成一棵树状结构的候选序列集合。作者在实验中发现，尽管预测“下一个” token 的 top-1 准确率仅约为 60%，但 top-5 的准确率却超过了 80%。这一显著提升表明，如果能够合理利用这些高概率预测结果，便可以在每个解码步骤中生成更多 token，从而显著加速推理过程。为了在 **不增加大模型调用次数** 的前提下高效验证多个候选路径，Medusa 引入了 **Tree Attention** 机制。



以一个具体例子说明：

假设当前有两个 Medusa 解码头：

- 第一个 head 输出 top-2 token: ["It", "I"]
- 第二个 head 输出 top-3 token: ["is", ",", "the"]

这就构成了 $2 \times 3 = 6$ 条可能路径：

路径 1: It → is
路径 2: It → '
路径 3: It → the
路径 4: I → is
路径 5: I → '
路径 6: I → the

我们将原始输入看作根节点，每条路径从根节点出发，经过 head1 的一个 token，再接上 head2 的一个 token，形成一个二层的候选树。

为了避免对每条路径逐个调用大模型进行验证，Medusa 使用 Tree Attention，将所有候选路径中的 token 扁平展开为一个连续序列：

```
["It", "I", "is", "'", "the", "is", "'", "the"]
```

- 前两个 token ("It", "I") 来自 head1；
- 后六个 token 是 head2 的三个候选，分别挂在 "It" 和 "I" 之下，因此每个出现两次。

接着，构建 **Tree Mask**，确保注意力机制只发生在逻辑上有因果关系的 token 之间：

- "It" 与其三个子节点 "is"、"'、"the" 可以相互注意；
- "I" 与其对应的子节点也可以相互注意；
- 但 "It" 和 "I" 不应该互相注意（它们是独立的路径）；
- 同样，head2 中不同父节点下的子 token 也不能互相注意；
- 只有沿树路径的祖先和后代 token 之间才允许注意力传递。

所以，利用 Tree Attention 就实现了一次大模型调用就把所有的路径都验证了的效果。考虑到真实场景，大模型的推理瓶颈往往在内存读取，而不是计算。所以适当地增加计算量，基本不会影响推理耗时，反而通过 Tree Attention 有可能得到更长的 draft。

7.3 Typical acceptance

在传统的 **Speculative Decoding** 中，最常用的候选验证方法是 **Rejection Sampling（拒绝采样）**。它的做法是：草稿模型（通常是一个小模型）先生成多个候选 token，原始大模型逐个验证这些 token 是否“符合自己概率分布”，不符合就全部丢弃。这种方式虽然准确，能保持输出分布和原始模型一致，但也存在一个严重问题——**效率低**。尤其当采样温度（temperature）较高时，草稿模型会生成更多富有创意、但偏离主流分布的候选 token。此时原始模型很容易拒绝它们，导致解码步骤缩短、重复计算，效率大大下降。

为了提高整体推理效率，**Medusa 提出了一种更宽容、更高效的验证策略 —— Typical Acceptance（典型接受）**。该策略从 **截断采样（Truncation Sampling）** 的研究中汲取灵感，目标是扩大原始模型可接受的候选范围。具体而言，Medusa 不再要求草稿模型生成的 token 与原始模型的分布完全对齐，而是根据原始模型的预测概率设定一个接受阈值：只要某个候选 token 的概率超过该阈值，就可以接受它以及它之前的所有 token（即其 prefix）。在这些被接受的 token 中，Medusa 使用贪婪策略（Greedy）选取 top-k，作为最终的候选输出。

它的核心思想是：Medusa 不强求草稿模型生成的 token 与原始模型的分布完全一致，只要这些 token 不是极不可能的结果，就可以被接受。

假设当前上下文是：“The weather is”，草稿模型（温度较高）生成的下一个 token 是：

```
"fun"
```

而原始模型的概率分布如下：

Token	概率
"nice"	0.35
"bad"	0.30
"cold"	0.15
"fun"	0.08
"wet"	0.06
"?"	0.03
"sad"	0.02
"quick"	0.01

我们设定一个累计概率阈值为 **90%**，从高到低累加：

- "nice" → 0.35
- "bad" → 0.65
- "cold" → 0.80

- "fun" → 0.88 
- "wet" → 0.94  超出阈值

Rejection Sampling 是怎么做的：

- draft 模型采样出 "fun" ；
- 原始模型判断 "fun" 的概率仅为 **0.08**（例如，草稿模型预测 "fun" 的概率为 0.8，而原始模型仅为 0.08，假设随机数 $r = 0.5$ ，由于 $r > 0.08 / 0.8 = 0.1$ ，因此该 token 被拒绝）。具体可参考第 2 小节中的拒绝采样示例。
- 然后重新采样，直到采到像 "nice" 或 "bad" 这样高概率的 token；
- 如果 draft 和原始模型温度不一致，这个过程会不断重试，效率变低。

Typical Acceptance 是怎么做的：

- 先构造出前 90% 累积概率所覆盖的 token 集合，称为 **典型 token 集合**；
- "fun" 虽然不是前几名，但累积后刚好落入这个集合中；
- 因此，它被认为“足够典型”，**直接接受，无需重采**；
- 如果后续 token 也在典型集合中，那么整个草稿序列都可以接受；
- 最终，我们选取最长被接受的前缀作为下一步解码结果。

7.4 Medusa 的缺点

每个 Medusa Head 是独立执行的，即“预测下下个 token”不会依赖于“下一个 token”的预测结果，缺乏序列间的依赖性，导致生成效果不佳、草稿接受率较低。

8 EAGLE

论文：

EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty: <https://arxiv.org/pdf/2401.15077>

EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees: <https://arxiv.org/pdf/2406.16858>

EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test: <https://arxiv.org/pdf/2503.01840>

Github: <https://github.com/SafeAILab/EAGLE>

EAGLE（Extrapolation Algorithm for Greater Language-model Efficiency）提出了一种新颖的推测采样框架，其核心创新在于：

- 在特征层级而非传统的 token 层级进行自回归预测。特征序列比 token 序列更具规律性，使得预测更为简单有效。
- 通过引入提前一步的 token 序列，解决了特征预测中的不确定性问题，从而提升预测的准确性和草稿生成质量。
- 该方法无需对目标 LLM 进行微调，保证生成文本的分布与传统自回归解码一致，实现无损加速。

EAGLE 经第三方评估认证，是目前最快的 Speculative Decoding 方法：

- 比常规 Speculative Decoding 快 3 倍（在 13B 模型上）；
- 比 Lookahead 快 2 倍（在 13B 模型上）；
- 比 Medusa 快 1.6 倍（在 13B 模型上）；



在 vLLM 中，可以通过如下方式启用 EAGLE。需要注意的是，EAGLE 的草稿模型需在**不启用张量并行**的模式下运行，而目标模型则可以正常使用张量并行以提升推理效率。

```
from vllm import LLM, SamplingParams

prompts = [
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    tensor_parallel_size=4, # 张量并行
    speculative_config={
        "model": "yuhui/EAGLE-LLaMA3-Instruct-8B",
        "draft_tensor_parallel_size": 1, # 无张量并行
    },
)

outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
```

```
generated_text = output.outputs[0].text
print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

8.1 核心思路

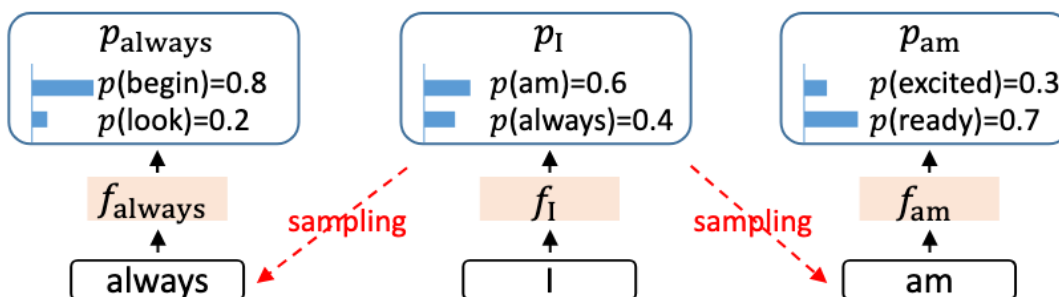
EAGLE 的作者提出了两个核心观点：

核心观点一：在特征层进行自回归比在 token 层进行自回归更简单

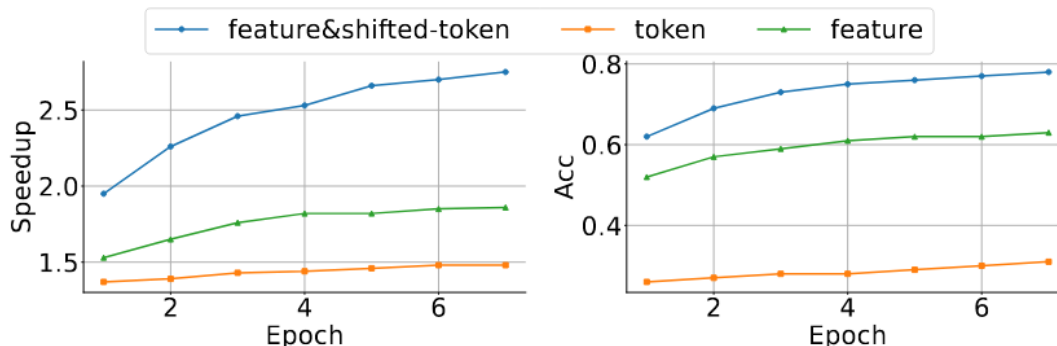
- **解释：**论文中的「特征」指的是 LLM 倒数第二层的输出，也就是在进入 LM Head 之前的隐藏状态 (hidden state)。相比于 token 序列，特征序列具有更强的规律性。
- **方案：**采用一个轻量级的自回归模型来预测目标模型的特征序列，而非直接预测 token。预测出的特征再通过目标模型的 LM Head 转换为 token 概率分布，用于后续采样生成。这种方法比直接预测 token 更容易，且效果更好。

核心观点二：采样过程中的不确定性限制了特征预测的性能

- **解释：**在文本生成过程中，LLM 的输出是带有随机性的，因为 LLM 会对 token 的概率分布进行采样。这种随机性导致特征序列的预测变得不确定。例如，给定相同的输入「I」，接下来可能出现「always」或者「am」，这就导致了特征预测的不确定性。



- **方案：**EAGLE 通过引入一个时间步长提前的 token 序列来解决这个问题。也就是说，在预测当前特征时，不仅考虑之前的特征序列，还考虑之前已经采样的 token 序列，从而减少特征预测的不确定性。下图表示，通过解决这个不确定性问题，EAGLE 的加速比从 1.9 倍进一步提升到 2.8 倍。



8.2 实现细节

EAGLE 的整体框架如下：

草稿阶段 (Drafting Phase) :

- **输入：**前一个时间步的特征序列，以及提前一步的 token 序列。
- **处理流程：**
 - 将 token 序列转换为对应的 embedding 向量序列。
 - 将 embedding 序列与特征序列在维度上拼接。
 - 使用一个自回归头 (Autoregression Head) 来预测下一个特征。
 - 使用 LM Head 将该特征映射为 token 的概率分布，并从中采样出下一个 token。
 - 将预测得到的特征与采样的 token 添加至输入序列中，继续下一轮预测。
- **输出：**一棵由多个候选 token 构成的草稿树 (draft tree) 。

验证阶段 (Verification Phase) :

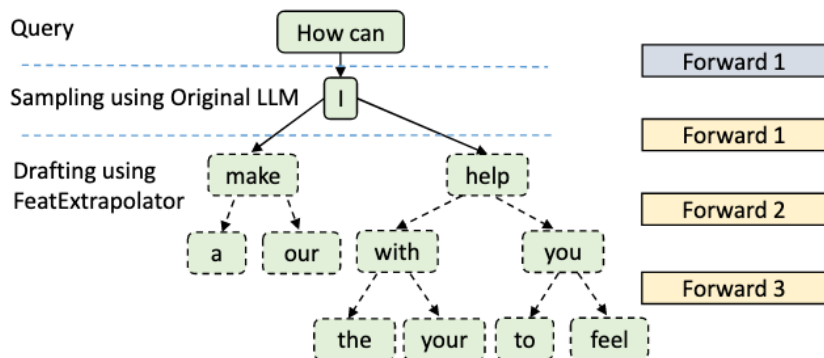
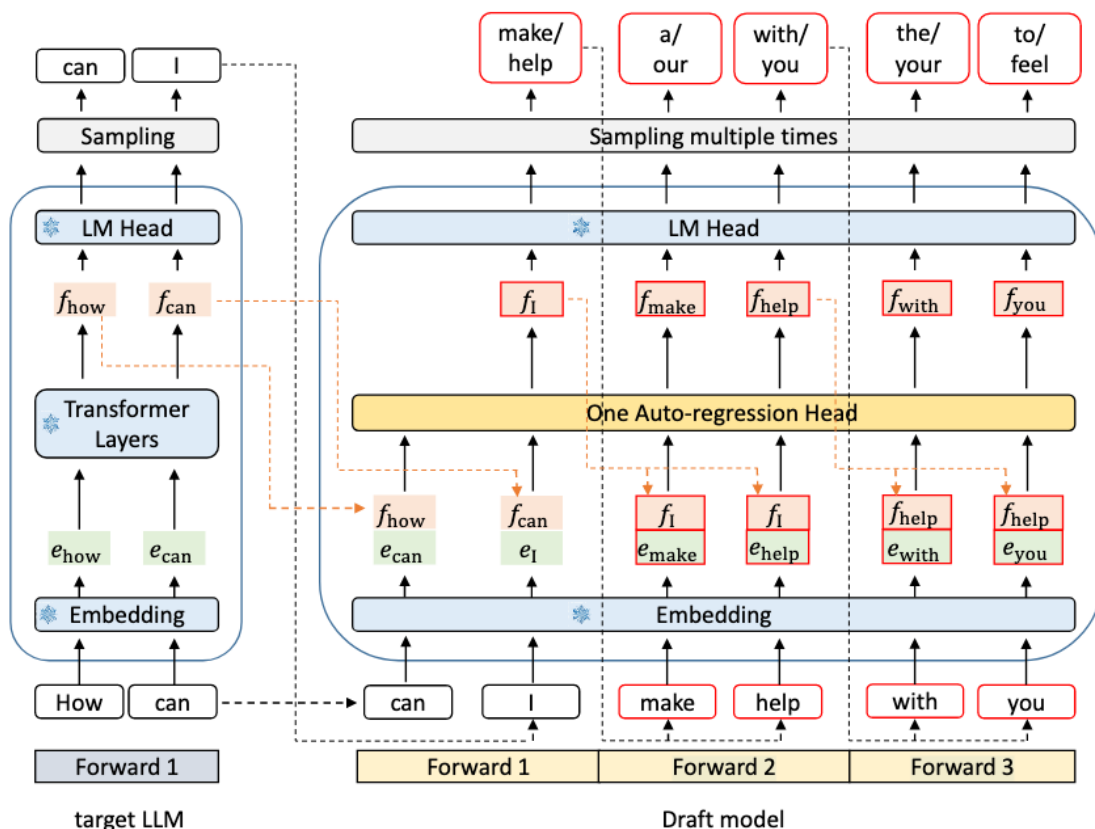
- **输入：**草稿树。
- **处理流程：**使用目标 LLM 对草稿树中的 token 逐一进行验证，判断其是否符合原始模型的分布。

- **输出：** 最终被接受的 token 序列，作为模型输出的一部分。

下图展示了 EAGLE 如何通过特征层级的自回归模型高效生成草稿 token 序列。整个 Draft Model（草稿模型）主要由三部分组成：**Embedding 层、Autoregression Head、LM Head**。其中，Embedding 和 LM Head 都复用目标 LLM 的参数，无需训练；唯一需要训练的是 Autoregression Head，由一个全连接层（FC）和一个 decoder 层组成。

整体流程如下：

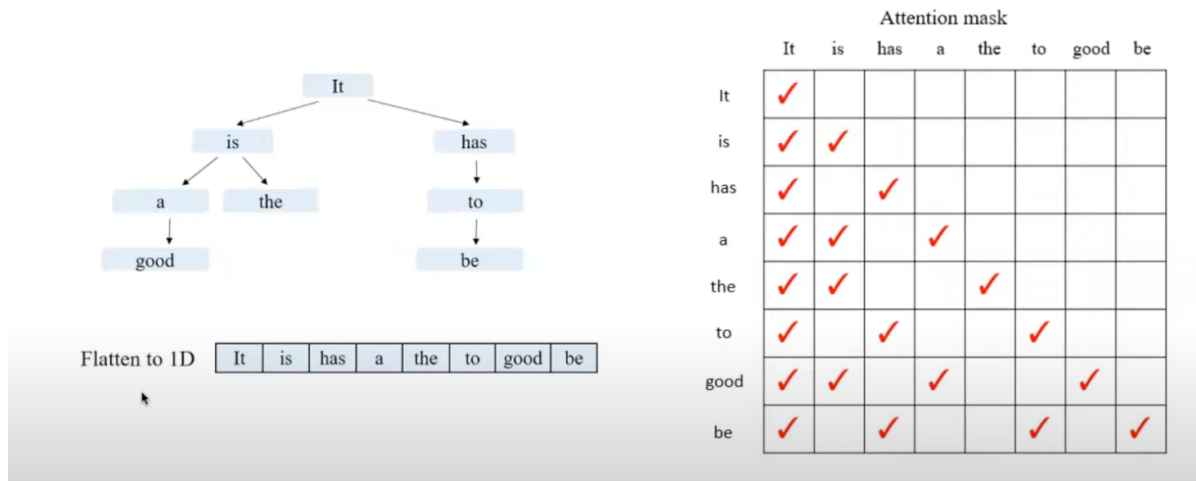
1. 假设当前已生成的 token 为 "How", "can", "I"。从原始 LLM 中取出 token "How" 和 "can" 在最后一层的输出特征向量，分别记作 f_{How} 、 f_{can} ，以及 token "can" 和 "I" 在输入端的 embedding 向量，分别为 e_{can} 、 e_{I} 。将 f_{How} 与 e_{can} 拼接， f_{can} 与 e_{I} 拼接，构成输入序列。
2. 拼接后的序列输入到 Autoregression Head，经过降维和解码器处理，预测出 token "I" 的特征向量 f_{I} 。然后将 f_{I} 输入到目标 LLM 的 LM Head，得到下一个 token 的分布，并从中采样出 token，例如 "make" 或 "help"。
3. 将采样得到的新 token（如 "make" 或 "help"）送入下一轮 forward，提取其 embedding 向量（如 e_{make} 、 e_{help} ），与上一轮的预测特征 f_{I} 拼接，作为下一步的输入，继续预测下一步的特征（如 f_{make} 、 f_{help} ），再由 LM Head 映射为 token（如 "a"、"our" 或 "with"、"you"）。
4. 每轮 forward 并非只预测一个 token，而是基于当前所有路径并行扩展多个分支。例如："make" 预测出 "a" 或 "our"，"help" 预测出 "with" 或 "you"。第三轮继续扩展为 "the"、"your"、"to"、"feel"，形成一棵 token 草稿树。



补充一点，第一次前向传播无法加速，因为需要通过一次前向传播才能得到后续 EAGLE 所需要的特征。

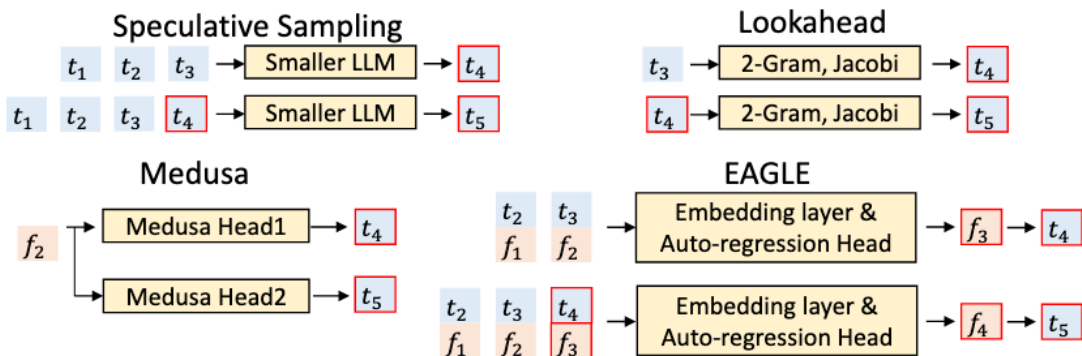
为了一次验证多个序列，EAGLE 采用了 Tree Attention 来生成树状结构的草稿，这样可以在一个前向传播过程中生成多个 token。

Tree attention



8.3 Speculative Sampling、Lookahead、Medusa、EAGLE 对比

EAGLE 论文中用下图对比了 Speculative Sampling、Lookahead、Medusa、EAGLE 不同方法生成草稿的对比。



- Speculative Sampling**: 使用一个较小的草稿模型快速生成多个 token，然后由目标 LLM 对这些 token 并行验证。该方法依赖草稿模型的质量，若草稿不佳，验证代价高，整体提速有限。
- Lookahead**: 利用 n-gram 或 Jacobi 等启发式策略，基于已生成的 token 推测未来 token。该方法无需引入新模型，延迟低，但草稿质量较差，通常仅适用于贪婪解码场景。
- Medusa**: 通过多个 MLP Head，对目标 LLM 倒数第二层的隐藏特征 (second-to-top-layer feature) 进行并行预测，生成多个候选 token。例如使用 f_2 同时预测 t_4 和 t_5 。尽管具备并行生成能力，但该方法无法捕捉 token 间的上下文依赖动态，导致生成分布与目标 LLM 偏离，尤以非贪婪采样场景下最为明显。
- EAGLE**: EAGLE 创新性地投机解码前移至特征层，使用目标模型自身输出的隐藏特征作为草稿模型输入，进行特征层级的自回归预测 (Autoregressive Decoding)。草稿模型与目标模型共享 embedding 层与 LM Head，仅新增一个轻量级的 Auto-regression Head。具体而言，如图所示，EAGLE 将 (f_1, e_2) 与 (f_2, e_3) 拼接后输入 Auto-regression Head 预测出 f_3 ，再通过 LM Head(f_3) 得到 token t_4 。

9 总结

推测解码 (Speculative Decoding) 是一种通过预生成多个候选 token 并并行验证以加速 LLM 推理的技术，旨在突破传统自回归解码中的内存带宽瓶颈。本文系统介绍了从早期草稿模型方法、Prompt Lookup 到 Jacobi Decoding、Lookahead、Medusa，再到当前速度领先的 EAGLE 等多种方案。尽管各方案实现方式不同，它们的共同目标是提升解码效率、降低推理延迟，同时保证生成文本的质量。

10 附录

10.1 什么是 Logits?

在大语言模型 (LLM) 中，logits 指的是模型输出层的原始得分向量，这些数值是未经过归一化处理的实数，表示模型对每个 token 作为一个生成 token 的“置信度”或“倾向性”。具体来说：

- logits 是模型最后一层 (通常是一个线性变换层) 的输出，向量的每个元素对应词表中一个 token 的得分，这些得分可以是正数或负数，范围从负无穷到正无穷。词表是训练语言模型时定义好的，包含模型可能生成的所有 token。假设当前的上下文是 "This apple"，要预测下一个 token：

```
# 假设词表:
vocab = ["This", "apple", "is", "very", "delicious", "bad", "today"]

# 模型输出的 logits (一个向量, 对应词表中每个 token 的得分):
logits = [1.8, 2.0, 2.5, 1.2, 0.5, 0.1, -0.7]
```

- 这些得分本身不是概率, 需要通过 softmax 函数将 logits 转换成概率分布, 概率值介于 0 和 1 之间且总和为 1, 从而确定生成下一个词的的概率。softmax 公式如下:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

计算每个元素的指数:

$$\exp(\text{logits}) = [e^{1.8}, e^{2.0}, e^{2.5}, e^{1.2}, e^{0.5}, e^{0.1}, e^{-0.7}] \approx [6.05, 7.39, 12.18, 3.32, 1.65, 1.11, 0.50]$$

求指数和:

$$\sum_{i=1}^7 e^{x_i} = 6.05 + 7.39 + 12.18 + 3.32 + 1.65 + 1.11 + 0.50 = 32.20$$

最终 softmax 概率为:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \Rightarrow \left[\frac{6.05}{32.20}, \frac{7.39}{32.20}, \frac{12.18}{32.20}, \frac{3.32}{32.20}, \frac{1.65}{32.20}, \frac{1.11}{32.20}, \frac{0.50}{32.20} \right] \approx [0.188, 0.229, 0.378, 0.103, 0.051, 0.034, 0.016]$$

- logits 值越大, 表示模型认为该词作为下一个词的可能性越高; 值越小, 可能性越低。

Token	Logit	Softmax概率
"This"	1.8	0.188
"apple"	2.0	0.229
"is"	2.5	0.378  (最高)
"very"	1.2	0.103
"delicious"	0.5	0.051
"bad"	0.1	0.034
"today"	-0.7	0.016

总的来说, logits 是 LLM 中预测下一个词的“未归一化概率得分”, 是生成过程中的关键中间表示, 决定了模型如何选择下一个输出 token。
logits 的长度等于词表 (vocabulary) 的大小。

10.2 雅可比迭代法 (Jacobi method)

雅可比迭代法 (Jacobi method) 是一种用于求解线性方程组的迭代算法。它通过迭代地更新方程组中每个未知数的近似值, 逐步逼近真实的解。这种方法特别适用于大型、稀疏的线性方程组。

雅可比迭代法的核心思想是将复杂的线性方程组分解为简单的对角部分和非对角部分, 然后利用当前迭代值计算下一次迭代值, 实现并行计算。当矩阵满足对角占优条件时 (即对角元素的绝对值大于同行其他元素绝对值之和), 不论初始值如何选取, 迭代过程都会收敛到唯一解, 因为每次迭代都会使误差逐步减小。

10.2.1 公式

给定一个 $n \times n$ 的线性方程组:

$$Ax = b$$

其中:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

将矩阵 A 分解为对角矩阵部分 D 与其余部分 R :

$$A = D + R \quad \text{其中} \quad D = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix}, \quad R = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ a_{21} & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{bmatrix}$$

此时线性方程可以变形为：

$$Dx = b - Rx$$

即将原方程拆成两部分，方便利用已知的 x 去迭代求解。

迭代表达式：

$$x^{(k+1)} = D^{-1}(b - Rx^{(k)})$$

表示使用当前第 k 次迭代的结果，去计算第 $k+1$ 次的近似解。

也可以写为单元素更新公式，可以直接看到第 i 个未知量如何受其他分量影响：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

10.2.2 具体例子

以一个 3×3 线性方程组为例：

$$\begin{cases} 4x_1 + x_2 + x_3 = 6 \\ x_1 + 5x_2 + x_3 = 7 \\ x_1 + x_2 + 6x_3 = 8 \end{cases}$$

用矩阵形式表示为：

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 1 & 5 & 1 \\ 1 & 1 & 6 \end{bmatrix}, \quad b = \begin{bmatrix} 6 \\ 7 \\ 8 \end{bmatrix}$$

应用雅可比迭代法的单元素更新公式：

对于 x_1 ：

$$x_1^{(k+1)} = \frac{1}{a_{11}} \left(b_1 - \sum_{j \neq 1} a_{1j} x_j^{(k)} \right) = \frac{1}{4} \left(6 - 1 \cdot x_2^{(k)} - 1 \cdot x_3^{(k)} \right)$$

对于 x_2 ：

$$x_2^{(k+1)} = \frac{1}{a_{22}} \left(b_2 - \sum_{j \neq 2} a_{2j} x_j^{(k)} \right) = \frac{1}{5} \left(7 - 1 \cdot x_1^{(k)} - 1 \cdot x_3^{(k)} \right)$$

对于 x_3 ：

$$x_3^{(k+1)} = \frac{1}{a_{33}} \left(b_3 - \sum_{j \neq 3} a_{3j} x_j^{(k)} \right) = \frac{1}{6} \left(8 - 1 \cdot x_1^{(k)} - 1 \cdot x_2^{(k)} \right)$$

假设初始值为：

$$x^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

进行迭代计算：

第一次迭代：

$$x_1^{(1)} = \frac{1}{4}(6 - 0 - 0) = 1.5$$

$$x_2^{(1)} = \frac{1}{5}(7 - 0 - 0) = 1.4$$

$$x_3^{(1)} = \frac{1}{6}(8 - 0 - 0) = 1.333$$

所以：

$$x^{(1)} = \begin{bmatrix} 1.5 \\ 1.4 \\ 1.333 \end{bmatrix}$$

第二次迭代：

$$x_1^{(2)} = \frac{1}{4}(6 - 1.4 - 1.333) = \frac{3.267}{4} \approx 0.817$$

$$x_2^{(2)} = \frac{1}{5}(7 - 1.5 - 1.333) = \frac{4.167}{5} \approx 0.833$$

$$x_3^{(2)} = \frac{1}{6}(8 - 1.5 - 1.4) = \frac{5.1}{6} \approx 0.85$$

所以：

$$x^{(2)} = \begin{bmatrix} 0.817 \\ 0.833 \\ 0.85 \end{bmatrix}$$

继续迭代，最终会收敛到精确解：

$$x = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

可以验证：

$$\begin{bmatrix} 4 & 1 & 1 \\ 1 & 5 & 1 \\ 1 & 1 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \neq \begin{bmatrix} 6 \\ 7 \\ 8 \end{bmatrix}$$

这个例子展示了雅可比迭代法如何通过反复应用单元素更新公式，逐步逼近三维线性方程组的解。

10.2.3 更通俗的例子

假设你、小明、小红三人一起玩一个猜数字游戏，每人猜一个数字，但你们的数字必须满足以下规则：

你的数字：

$$x_{\text{你}} = \frac{6 - x_{\text{小明}} - x_{\text{小红}}}{4}$$

小明的数字：

$$x_{\text{小明}} = \frac{7 - x_{\text{你}} - x_{\text{小红}}}{5}$$

小红的数字：

$$x_{\text{小红}} = \frac{8 - x_{\text{你}} - x_{\text{小明}}}{6}$$

也就是说，每个人的数字都需要参考另外两个人的数字。

游戏过程（详细迭代示例）

第 0 轮（初始猜测）

刚开始，你们都没有头绪，于是都从 0 开始猜：

玩家	猜测数字
你	0
小明	0
小红	0

第 1 轮

每个人根据上轮其他人的猜测更新自己的数字：

你的新数字：

$$x_{\text{你}}^{(1)} = \frac{6 - 0 - 0}{4} = 1.5$$

小明的新数字：

$$x_{\text{小明}}^{(1)} = \frac{7 - 0 - 0}{5} = 1.4$$

小红的新数字：

$$x_{\text{小红}}^{(1)} = \frac{8 - 0 - 0}{6} = 1.333$$

第 2 轮

继续迭代，用第1轮的结果更新：

你的新数字：

$$x_{\text{你}}^{(2)} = \frac{6 - 1.4 - 1.333}{4} = 0.817$$

• 小明的新数字：

$$x_{\text{小明}}^{(2)} = \frac{7 - 1.5 - 1.333}{5} = 0.833$$

• 小红的新数字：

$$x_{\text{小红}}^{(2)} = \frac{8 - 1.5 - 1.4}{6} = 0.85$$

第 3 轮

继续迭代，用第2轮的结果更新：

你的新数字：

$$x_{\text{你}}^{(3)} = \frac{6 - 0.833 - 0.85}{4} = 1.079$$

小明的新数字：

$$x_{\text{小明}}^{(3)} = \frac{7 - 0.817 - 0.85}{5} = 1.067$$

小红的新数字：

$$x_{\text{小红}}^{(3)} = \frac{8 - 0.817 - 0.833}{6} = 1.058$$

第 4 轮

继续迭代，用第3轮的结果更新：

• 你的新数字：

$$x_{\text{你}}^{(4)} = \frac{6 - 1.067 - 1.058}{4} = 0.969$$

• 小明的新数字：

$$x_{\text{小明}}^{(4)} = \frac{7 - 1.079 - 1.058}{5} = 0.973$$

• 小红的新数字：

$$x_{\text{小红}}^{(4)} = \frac{8 - 1.079 - 1.067}{6} = 0.976$$

持续进行迭代（直到收敛）

随着迭代次数增加，你们的猜测会越来越稳定，最终逐渐收敛到：

玩家	最终猜测（趋近值）
你	1.0
小明	1.0
小红	1.0

此时，三个人的数字几乎不会再变动，游戏可以结束。

为什么游戏可以收敛到正确答案？

- **相互依赖**：每个人的数字取决于其他人的数字，类似于数学中线性方程组的变量间的关系。
- **信息传递**：每轮迭代，每个人都参考了上一轮其他人的结果，不断修正自身猜测。
- **并行更新**：所有人都同时进行更新，这正是 Jacobi 迭代法的特点之一。
- **必然收敛**：由于满足对角占优条件，无论初始值如何选择，最终都会收敛到相同的答案。

这种猜数字游戏不仅仅是个数学游戏，还能反映现实世界中很多场景，比如：

- **交通网络流量**：每个路口的流量都会受到相邻路口的影响，最终形成稳定的车流状态。
- **经济市场均衡**：市场中的每个参与者的行为受到其他人的行为影响，经过多次博弈后达到市场均衡。

Jacobi 迭代法的核心，就是不断应用简单的**局部更新规则**（局部信息），最终达到**全局平衡状态**（方程组解）。

10.3 Jacobi Decoding 详解

假设目标输入：

```
x = ["Alan", "Turing"]
```

最终想生成的理想 token 序列是：

```
["who", "was", "a"]
```

Step 0: 初始猜测

假设给定一个初始不太靠谱的初始猜测序列：

y^{(0)} = [the, computer, engineer]

Step 1: 根据 y^{(0)} 并行计算 y^{(1)}

我们根据 y^{(0)} 的前缀依次预测每个 token：

- y_1^{(1)} = LM(["Alan", "Turing"]) = "who" ✓ 修正了第一个 token
- y_2^{(1)} = LM(["Alan", "Turing", "the"]) = "is" ⚠ 没有直接修正为 "was"，但比 "computer" 更合理
- y_3^{(1)} = LM(["Alan", "Turing", "the", "computer"]) = "pioneer" ✗ 仍未对，但更相关

因此：

y^{(1)} = [who, is, pioneer]

Step 2: 使用 y^{(1)} 继续迭代

此轮中，所有预测都基于 y^{(1)}：

- y_1^{(2)} = LM(["Alan", "Turing"]) = "who" ✓ 收敛，继续保持
- y_2^{(2)} = LM(["Alan", "Turing", "who"]) = "was" ✓ 修正了 "is"
- y_3^{(2)} = LM(["Alan", "Turing", "who", "is"]) = "a" ✓ 虽然前缀非目标 "was"，但仍然成功预测出正确 token！

得到：

y^{(2)} = [who, was, a]

与目标完全一致，收敛！

流程总结：

步骤	y ₁	y ₂	y ₃	说明
y ⁰	the	computer	engineer	初始猜测，完全错误
y ¹	who	is	pioneer	第一轮，部分修正
y ²	who	was	a	第二轮，完全收敛

11 参考资料

• Medusa: Simple Framework for Accelerating LLM Generation with Multiple Decoding Heads: <https://sites.google.com/view/medusa-llm>

• Speculative Sampling — Intuitively and Exhaustively Explained: <https://medium.com/intuitively-and-exhaustively-explained/speculative-sampling-intuitively-and-exhaustively-explained-2daca347dbb9>

• Logits as Confidence: The Hidden Power AI Engineers Need to Unlock in LLMs and VLMs: <https://medium.com/@adkananthi/logits-as-confidence-the-hidden-power-ai-engineers-need-to-unlock-in-llms-and-vlms-194d512c31f2>

• 大模型推理妙招—投机采样（Speculative Decoding）: <https://zhuanlan.zhihu.com/p/651359908>

• 探秘Transformer系列之（31）--- Medusa: https://blog.csdn.net/weixin_47364682/article/details/147569594

• Speculative Decoding and Beyond: A Survey of Speculative Decoding Techniques: <https://blog.codingconfessions.com/p/a-selective-survey-of-speculative-decoding>

• EAGLE: Extrapolation Algorithm for Greater Language-model Efficiency: <https://sites.google.com/view/eagle-llm>

• Assisted Generation: a new direction toward low-latency text generation: <https://huggingface.co/blog/assisted-generation>

• [V1][Spec Decode] Ngram Spec Decode: <https://github.com/vllm-project/vllm/pull/12193>

• apoorvumang/prompt-lookup-decoding: <https://github.com/apoorvumang/prompt-lookup-decoding>

• 探秘Transformer系列之（32）--- Lookahead Decoding: <https://www.cnblogs.com/rossiXYZ/p/18859771>

- Medusa: Simple Framework for Accelerating LLM Generation with Multiple Decoding Heads: <https://sites.google.com/view/medusa-llm>
- 【论文解读】EAGLE: 在特征层进行自回归的投机采样框架: <https://zhuanlan.zhihu.com/p/15955544919>
- 万字综述 10+ 种 LLM 投机采样推理加速方案: <https://mp.weixin.qq.com/s/PyAKiFzbQNq6w7HmaTnSEw>
- EAGLE and EAGLE-2: Lossless Inference Acceleration for LLMs - Hongyang Zhang: <https://www.youtube.com/watch?v=oXRSorx-Llg>
- DeepSeek-V3 MTP 工程实现思考: <https://zhuanlan.zhihu.com/p/29082207943>
- 探秘 Transformer 系列之 (33) --- DeepSeek MTP: <https://www.cnblogs.com/rossiXYZ/p/18880573>
- Accelerating Large Language Models: A Deep Dive into Speculative Decoding and its vLLM Implementation: <https://medium.com/@abhinaykrishna/accelerating-large-language-models-a-deep-dive-into-speculative-decoding-and-its-vllm-9208e8e6e6c6>
- Speculative Decoding 论文阅读合订本: <https://zhuanlan.zhihu.com/p/684217993>
- Andrej Karpathy X: <https://x.com/karpathy/status/1697318534555336961>
- Lookahead Decoding 图文详解: <https://zhuanlan.zhihu.com/p/701015670>